DTD BATTLEGROUND - PROJECT STATUS

Date: 19 October 2026

EXECUTIVE SUMMARY

DTD Battleground is a working, single-event quiz platform for the JNTUH Data Science Club. It supports team registration and login, three quiz rounds, an organiser dashboard, a live display, server-authoritative scoring, question secrecy, backups, and persistent state.

The current performance hardening work is on branch async-registration-persistence.

ARCHITECTURE

Client:
- React 19 + Vite frontend.
- Team app: /; organiser dashboard: /#/admin; projector display: /#/display.
- The browser stores only team/admin session tokens. It never receives question answer keys.

Development:
- Vite runs on port 5173 and proxies /api/* to the Node server on port 3001.

Event deployment:
- After npm run build, npm start serves both the built frontend and API from port 3001.

Backend:
- Node HTTP server owns rules, timers, authentication, grading, standings, buzzer flow, and admin actions.
- Questions and answers are loaded from server/questions.json; sample questions are used only when the real file is absent.
- Authoritative game state is held in memory and persisted as server/data/state.json.

PRODUCT CAPABILITIES

- Team registration, passcodes, up to three members, one active login per named member.
- Practice round, timed Round 1, market/solve Round 2, and host-controlled buzzer final.
- Server-side timing and answer validation.
- Organiser phase control, score corrections with audit reasons, tie breaks, finalist locking, CSV standings, and JSON backup.
- Projector-safe display and private team views.

QUALITY CHECKS

- Unit tests: 10/10 passing (npm test).
- Production frontend build: passing (npm run build).
- HTTP smoke test: registration issued a token, state appeared through the API, and queued persistence reached disk.

LOAD-TEST EVIDENCE

Read-only state polling

A k6 ramp to 1,000 virtual users polling /api/state completed with 0% HTTP failures. Results included p95 2.77 ms and p99 5.21 ms. This validates the lightweight public polling path on the test laptop.

Registration burst before hardening

The initial 150-VU repeated-registration test caused multi-second delays and occasional failures. The main causes were synchronous scryptSync password hashing and synchronous full-state disk writes on the Node event loop.

Registration burst after hardening

The 150-VU repeated-registration test achieved:
- 1,639 successful registrations in about 31.4 seconds (52.26 registrations/second).
- 0% registration failures and 0% HTTP failures.
- Registration p95: 1.61 seconds.
- Polling p95 while registrations were running: 786 ms.

This is about 3x the successful registration throughput of the prior implementation. It remains an intentionally harsher workload than event check-in because every virtual user continually creates new teams; the run created more than 1,600 fake teams.

CURRENT BRANCH CHANGES

Branch: async-registration-persistence

Commits:
- 6205802 Improve registration burst handling
- ac279f3 Handle malformed HTTP request URLs

Implemented improvements:
- HTTP registration and login now use asynchronous crypto.scrypt() with await, so hashing runs in Node’s worker pool rather than freezing timers and polling on the event loop.
- State persistence is coalesced for 250 ms and serialized. There is at most one disk write in flight and one dirty-state flag; requests do not create an unbounded write

queue.
- Each persistence operation writes a unique temporary file and atomically renames it to state.json.
- Shutdown handling flushes pending persistence before normal process exit.
- The HTTP request boundary now returns HTTP 400 for malformed request URLs rather than crashing the request handler.

KNOWN RISKS AND RECOMMENDED NEXT WORK

1. Persistence health: a failed temp-file write or rename preserves the old valid state file, so atomicity is retained, but the newest state may not be durable. Add persistence health status, an admin-visible warning, exponential retry backoff, and stale temporary-file cleanup.

2. Hashing backpressure: asynchronous hashing avoids blocking the event loop but does not make hashing free. Add a bounded global hash/registration queue with a small concurrency limit and a 503/Retry-After response when saturated. Do not blindly increase UV_THREADPOOL_SIZE without retesting CPU and memory on the event laptop.

3. Realistic benchmark: add a k6 scenario where each participant registers only once and then polls. Test 50 and 75 teams on the actual event laptop and venue router. The existing 150-VU continuous registration script is a useful breaking-point test, not a check-in simulation.

4. Large-event scaling: for several hundred active teams, move sessions/state to Redis and durable data to a database, and replace frequent polling with WebSockets or Server-Sent Events.

READINESS ASSESSMENT

The platform is ready for the intended small-to-medium club event after normal operational testing. The current branch materially improves burst resilience and has no observed request failures in the latest aggressive k6 test.

It is not yet proven to meet a sub-second p95 latency target under continuous 150-user registration overload. Before merging to main, implement bounded hashing backpressure and persistence-health reporting, then rerun the realistic registration benchmark.